

FourSeasonsLib Unit Testing using NUnit

Introduction

This experiment explains how to write unit test cases for the FourSeasonsLib project using the NUnit framework. The application determines the season based on the given month. TestCaseSource is used to execute multiple test cases with minimum code.

Objectives

- Learn automated unit testing using NUnit.
- Understand the TestCaseSource attribute.
- Reduce duplicate code by using a common data source.
- Verify season mapping for different months using Assert.That().

Implementation

Step 1: Create a Class Library project named FourSeasonsLib.Tests in the same solution.

Step 2: Rename the default class to FourSeasonsTests.cs.

Step 3: Add the reference of FourSeasonsLib to the test project.

Step 4: Install NUnit, NUnit3TestAdapter and Moq using NuGet Package Manager.

Step 5: Create a data source containing different month names and their expected seasons.

Step 6: Use the TestCaseSource attribute to read test data from the data source.

Step 7: Write a single test method to validate all month and season combinations.

Step 8: Use Assert.That() to compare the expected season with the actual season.

Step 9: Build the solution and execute all test cases.

Step 10: Modify the season logic intentionally to break a test.

Step 11: Run the tests again and observe the failed test result.

Execution

Paste the screenshot of the successful NUnit Test Explorer or terminal output here.

The screenshot shows a Visual Studio IDE with a C# project named 'FourSeasonsLib'. The 'EXPLORER' pane on the left shows the project structure with files: 'bin', 'FourSeasonsLib.Tests', 'obj', 'FourSeasonsLib.Tests.csproj', 'SeasonConverter.cs', and 'SeasonConverterTests.cs'. The 'SeasonConverterTests.cs' file is open in the editor, showing a test class with a single test method 'GetSeason_ValidMonth_UsingDictionary'. The test method uses 'TestCasesSource' to iterate over a dictionary of months and seasons. The 'TERMINAL' pane at the bottom shows the command prompt output for running the tests. The output indicates that the tests were successful, with a summary showing 23 tests passed and 0 failed.

```
PS C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib> dotnet add FourSeasonsLib.Tests reference FourSeasonsLib\FourSeasonsLib.csproj
>> dotnet add FourSeasonsLib.Tests package NUnit
>> dotnet add FourSeasonsLib.Tests package NUnit3TestAdapter
>> dotnet add FourSeasonsLib.Tests package Microsoft.NET.Test.Sdk
>> dotnet add FourSeasonsLib.Tests package Moq --
Info : CACRT https://api.nuget.org/v3-vulnerabilities/2026.07.16.05.27.22/2026.07.17.11.27.26/vulnerability.update.json
Info : Package 'Moq' is compatible with all the specified frameworks in project 'C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib\FourSeasonsLib.Tests\FourSeasonsLib.Tests.csproj'.
Info : PackageReference for package 'Moq' version '4.20.72' added to file 'C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib\FourSeasonsLib.Tests\FourSeasonsLib.Tests.csproj'.
Info : Writing assets file to disk. Path: C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib\FourSeasonsLib.Tests\obj\project.assets.json
log : Restored C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib\FourSeasonsLib.Tests\FourSeasonsLib.Tests.csproj (in 220 ms).
PS C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib> dotnet test

Restore complete (0.6s)
FourSeasonsLib.Tests net8.0 succeeded with 1 warning(s) (1.1s) → bin\Debug\net8.0\FourSeasonsLib.Tests.dll
C:\Program Files\dotnet\sdk\10.0.301\Microsoft.Common.CurrentVersion.targets(2205,5): warning MSB99008: The referenced project ..\FourSeasonsLib\FourSeasonsLib.csproj does not exist.
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib\bin\Debug\net8.0\FourSeasonsLib.Tests.dll
NUnit3TestExecutor discovered 23 of 23 NUnit test cases using Current Discovery mode, Non-Explicit run
NUnit Adapter 4.5.0.0: Test execution complete
FourSeasonsLib.Tests test net8.0 succeeded (1.4s)

Test summary: total: 23, failed: 0, succeeded: 23, skipped: 0, duration: 1.4s
Build succeeded with 1 warning(s) in 3.9s
PS C:\Users\VASUS\OneDrive\Desktop\FourSeasonsLib>
```

Expected Result

The application should correctly return Spring, Summer, Monsoon, Autumn, or Winter based on the input month. All TestCasesSource test cases should pass when the implementation is correct.

Conclusion

This experiment demonstrated how TestCasesSource helps reduce duplicate code by executing multiple test scenarios through a single test method. It improves readability and simplifies unit test maintenance.